

Meta Skills

Skills about skills — 用于管理、发现、创建和分析 Claude Code Skills 的元技能集合。

概述

Meta Skills 是一套围绕 Claude Code Skill 生态系统构建的工具集，帮助用户更高效地使用和管理 skills。它分为四大模块：

模块	说明	核心能力
Search	搜索 Skill	从本地和远程 skill 库中快速定位所需 skill
Recommend	推荐 Skill	根据用户上下文和意图智能推荐最合适的 skill
Create & Improve	创建/改进 Skill	从零构建新 skill，或对现有 skill 进行迭代优化
Analyze	分析 Skill	评估 skill 质量、性能、使用情况和改进空间

Skill 清单

详见 [SKILL_LIST.md](#) 获取完整清单。

Search — 搜索 Skill

Skill	链接	说明
skill-skillsh-finder	GitHub	基于 Shell 脚本的 skill 搜索工具
skill-find-skills-clawhub	GitHub	从 ClawHub 平台查找 skill

Recommend — 推荐 Skill

Skill	链接	说明
skill-recommender	GitHub	根据用户上下文智能推荐 skill

Create & Improve — 创建/改进 Skill

Skill	链接	说明
skill-master	GitHub	Skill 创建与改进工具

Analyze — 分析 Skill

Skill	链接	说明
plan-visualizer	GitHub	将计划、任务列表和工作流可视化交互式 HTML 页面
session-distiller	GitHub	分析对话内容，自动提取可复用的技能模式
skill-analyzer	GitHub	递归分析 skill 的完整依赖关系并生成交互式可视化图表

安装

本仓库是一个 Claude Code Plugin，包含全部 7 个 meta-skill。

从 Marketplace 安装

```
/plugin install meta-skills
```

手动安装

```
git clone --recurse-submodules https://github.com/intelli-train-ai/meta-skills.git
```

然后通过 `--plugin-dir` 加载：

```
claude --plugin-dir ./meta-skills
```

安装后可用的 Skill 命令

命令	说明
<code>/meta-skills:skill-skillsh-finder</code>	从 skills.sh 搜索 skill
<code>/meta-skills:skill-find-skills-clawhub</code>	从 ClawHub 平台查找 skill
<code>/meta-skills:skill-recommender</code>	根据上下文智能推荐 skill
<code>/meta-skills:skill-master</code>	创建与改进 skill

命令	说明
<code>/meta-skills:skill-plan-visualizer</code>	可视化计划和工作流
<code>/meta-skills:skill-session-distiller</code>	从对话中提取可复用技能模式
<code>/meta-skills:skill-analyzer</code>	分析 skill 依赖关系

相关资源

资源	链接	说明
skill-template	GitHub	Skill 模板，帮助用户快速编写新 skill

用户故事

找到合适的 Skill

"我想生成一个 API 文档，但不知道有没有现成的 skill 能用。"

用户通过 **skill-skillsh-finder** 或 **skill-find-skills-clawhub** 搜索关键词，快速找到匹配的 skill 并安装使用。

获得智能推荐

"我正在写一个 React 项目，有哪些 skill 能帮到我？"

skill-recommender 根据当前项目上下文（语言、框架、正在进行的任务）主动推荐最相关的 skill，无需用户手动搜索。

从零创建 Skill

"我有一个重复性工作流，想把它封装成 skill 分享给团队。"

skill-master 引导用户从模板开始，逐步构建、测试并发布新 skill。

分析和改进现有 Skill

"我的 skill 依赖关系越来越复杂，想理清清楚结构。"

skill-analyzer 递归分析依赖关系并生成可视化图表；**session-distiller** 从历史对话中提取可复用的模式，帮助持续迭代。

可视化工作计划

"我想把当前的任务拆解和执行计划分享给同事看。"

plan-visualizer 将计划和工作流导出为交互式 HTML 页面，方便团队协作和评审。

设计原则

- **组合优于单体**：每个 meta skill 专注做好一件事，通过组合实现复杂工作流
- **上下文感知**：尽可能利用当前工作上下文来提供更精准的结果
- **渐进式使用**：从简单搜索开始，按需深入到创建和分析
- **自举能力**：meta skills 本身也是 skills，可以用 meta skills 来改进 meta skills

Meta Skills 完整清单

共 7 个 meta skill，分为 4 大类。

1. Search — 搜索 Skill

帮助用户从 skill 库中找到所需的 skill。

Skill	链接	说明
skill-skillsh-finder	GitHub	基于 Shell 脚本的 skill 搜索工具
skill-find-skills-clawhub	GitHub	从 ClawHub 平台查找 skill

2. Recommend — 推荐 Skill

根据上下文和用户画像，主动推荐最合适的 skill。

Skill	链接	说明
skill-recommender	GitHub	根据用户上下文智能推荐 skill

3. Create & Improve — 创建/改进 Skill

从零构建新 skill，或对现有 skill 进行迭代优化。

Skill	链接	说明
skill-master	GitHub	Skill 创建与改进工具

4. Analyze — 分析 Skill

评估 skill 的质量、性能和变更影响。

Skill	链接	说明
plan-visualizer	GitHub	将计划、任务列表和工作流可视化为交互式 HTML 页面
session-distiller	GitHub	分析对话内容，自动提取可复用的技能模式
skill-analyzer	GitHub	递归分析 skill 的完整依赖关系并生成交互式可视化图表

相关资源

资源	链接	说明
skill-template	GitHub	Skill 模板，帮助用户快速编写新 skill

findskill Skill 实现文档

从自建 Skill 平台搜索、下载、发布 Claude Code Skills。

1. API 端点总览

方法	路径	说明
GET	/api/skills/search	按关键词/分类搜索 skill
GET	/api/skills/:id/download	下载 skill 文件包
POST	/api/skills	发布新 skill
GET	/api/categories	获取分类列表

2. 端点详细设计

2.1 搜索 Skill

GET /api/skills/search

请求参数（Query String）：

参数	类型	必填	说明
q	string	否	搜索关键词，匹配 name 和 description
category	string	否	分类筛选
tag	string	否	标签筛选，多个用逗号分隔
author	string	否	按作者筛选
sort	string	否	排序方式：downloads（默认） / newest / updated
page	int	否	页码，默认 1
per_page	int	否	每页数量，默认 20，最大 100

响应示例：

```
{
  "total": 128,
  "page": 1,
  "per_page": 20,
  "results": [
    {
      "id": "skill-pdf-tools",
      "name": "pdf-tools",
      "description": "PDF 读取、合并、拆分工具",
      "author": "orion",
      "category": "工具",
      "tags": ["pdf", "document"],
      "downloads": 1234,
      "created_at": "2026-01-15T08:00:00Z",
      "updated_at": "2026-03-20T12:00:00Z"
    }
  ]
}
```

2.2 下载 Skill

```
GET /api/skills/:id/download
```

路径参数：

参数	类型	说明
id	string	skill 唯一标识

响应：

返回 `.tar.gz` 压缩包（`Content-Type: application/gzip`），包含完整的 skill 目录结构：

```
skill-name/
├── SKILL.md
├── scripts/      # 可选
├── references/   # 可选
└── assets/       # 可选
```

响应头：

```
Content-Type: application/gzip
Content-Disposition: attachment; filename="skill-name.tar.gz"
```

错误响应：

```
// 404
{ "error": "skill_not_found", "message": "Skill 不存在" }
```

2.3 发布 Skill

POST /api/skills

请求头：

Content-Type: multipart/form-data
Authorization: Bearer <token>

请求字段 (multipart)：

字段	类型	必填	说明
name	string	是	skill 名称 (唯一标识)
description	string	是	skill 简介
category	string	是	所属分类
tags	string	否	标签, 逗号分隔
file	file	是	<code>.tar.gz</code> 压缩包, 包含 SKILL.md 及相关文件

响应示例：

```
// 201 Created
{
  "id": "skill-pdf-tools",
  "name": "pdf-tools",
  "message": "发布成功"
}
```

错误响应：

```
// 400
{ "error": "invalid_package", "message": "压缩包中缺少 SKILL.md" }

// 409
{ "error": "name_conflict", "message": "skill 名称已存在" }
```

```
// 401
{ "error": "unauthorized", "message": "缺少或无效的 token" }
```

2.4 分类列表

```
GET /api/categories
```

无请求参数。

响应示例：

```
{
  "categories": [
    { "name": "搜索", "count": 15 },
    { "name": "开发", "count": 42 },
    { "name": "工具", "count": 28 },
    { "name": "集成", "count": 19 },
    { "name": "效率", "count": 11 }
  ]
}
```

3. Skill workflow 设计

3.1 搜索流程

用户输入关键词

- GET /api/skills/search?q=xxx
- 格式化展示结果列表
- 用户选择某个 skill
- 询问安装位置（全局 / 项目）
- 执行下载安装

3.2 下载安装流程

```
GET /api/skills/:id/download
```

- 保存 .tar.gz 到临时目录
- 解压到目标位置：
 - 全局： ~/.claude/skills/<name>/
 - 项目： .claude/skills/<name>/
- 验证 SKILL.md 存在且 frontmatter 合法

- 清理临时文件
- 输出安装成功信息

3.3 发布流程

- 用户指定 skill 目录路径
- 验证目录结构 (SKILL.md 必须存在)
 - 从 SKILL.md frontmatter 提取 name、description
 - 打包为 .tar.gz
 - POST /api/skills (需要 token)
 - 输出发布结果

3.4 浏览分类流程

- GET /api/categories
- 展示分类列表及 skill 数量
 - 用户选择分类
 - GET /api/skills/search?category=xxx
 - 展示该分类下的 skill

4. 认证方式

操作	是否需要认证
搜索	否
下载	否
发布	是 (Bearer Token)
分类列表	否

Token 存储位置: `~/.findskill/token`, skill 首次发布时引导用户配置。

5. 错误码规范

HTTP 状态码	error 字段	说明
400	invalid_request	请求参数不合法
400	invalid_package	上传的压缩包格式不正确

HTTP 状态码	error 字段	说明
401	unauthorized	未认证或 token 无效
404	skill_not_found	skill 不存在
409	name_conflict	skill 名称冲突
429	rate_limited	请求过于频繁
500	internal_error	服务器内部错误